assign-day-15

July 21, 2023

1 Assignment Problems: Day 15

The following problems require access to the file named 'Dataset_Day15.csv', provided with this assignment.

The data contains transaction details with 7501 rows. Each row is a grocery transaction from the market with the unique items list for each transaction.

[2]:
```
import pandas as pd
from csv import reader
from mlxtend.frequent_patterns import apriori
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import association_rules
```

- Read the data and remove all null or empty values.

[3]:
```
groceries =[]
with open('Dataset_Day15.csv','r') as csvfile:
    read = reader(csvfile)
    for row in read:
        row_new = [x for x in row if x!='NaN' and x!='']
        groceries.append(row_new)
groceries[0:5]
```

[3]:
```
[['shrimp',
  'almonds',
  'avocado',
  'vegetables mix',
  'green grapes',
  'whole weat flour',
  'yams',
  'cottage cheese',
  'energy drink',
  'tomato juice',
  'low fat yogurt',
  'green tea',
  'honey',
  'salad',
  'mineral water',
```

```
 'salmon',
 'antioxydant juice',
 'frozen smoothie',
 'spinach',
 'olive oil'],
['burgers', 'meatballs', 'eggs'],
['chutney'],
['turkey', 'avocado'],
['mineral water', 'milk', 'energy bar', 'whole wheat rice', 'green tea']]
```

- Transform the data using TransactonEncoder to perform Market Basket Analysis.

[4]:
```
encoder = TransactionEncoder()
transactions = encoder.fit(groceries).transform(groceries)
itemsets = pd.DataFrame(transactions, columns = encoder.columns_)
itemsets
```

[4]:

	asparagus	almonds	antioxydant juice	asparagus	avocado	babies food	\
0	False	True	True	False	True	False	
1	False	False	False	False	False	False	
2	False	False	False	False	False	False	
3	False	False	False	False	True	False	
4	False	False	False	False	False	False	
…	…	…	…	…	…	…	
7496	False	False	False	False	False	False	
7497	False	False	False	False	False	False	
7498	False	False	False	False	False	False	
7499	False	False	False	False	False	False	
7500	False	False	False	False	False	False	

	bacon	barbecue sauce	black tea	blueberries	…	turkey	\
0	False	False	False	False	…	False	
1	False	False	False	False	…	False	
2	False	False	False	False	…	False	
3	False	False	False	False	…	True	
4	False	False	False	False	…	False	
…	…	…	…	…	…	…	
7496	False	False	False	False	…	False	
7497	False	False	False	False	…	False	
7498	False	False	False	False	…	False	
7499	False	False	False	False	…	False	
7500	False	False	False	False	…	False	

	vegetables mix	water spray	white wine	whole weat flour	\
0	True	False	False	True	
1	False	False	False	False	
2	False	False	False	False	

```
3              False        False       False             False
4              False        False       False             False
…              …          …           …                 …
7496           False        False       False             False
7497           False        False       False             False
7498           False        False       False             False
7499           False        False       False             False
7500           False        False       False             False

      whole wheat pasta  whole wheat rice   yams  yogurt cake  zucchini
0                 False             False   True        False     False
1                 False             False  False        False     False
2                 False             False  False        False     False
3                 False             False  False        False     False
4                 False              True  False        False     False
…                 …                …      …          …       …
7496              False             False  False        False     False
7497              False             False  False        False     False
7498              False             False  False        False     False
7499              False             False  False        False     False
7500              False             False  False         True     False

[7501 rows x 120 columns]
```

[5]:
```
itemsets.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7501 entries, 0 to 7500
Columns: 120 entries,  asparagus to zucchini
dtypes: bool(120)
memory usage: 879.2 KB
```

- Use min_support = 0.02 to find frequent itemsets.

[6]:
```
frequent_itemsets = apriori(itemsets, min_support = 0.02, use_colnames = True)
frequent_itemsets['length'] = frequent_itemsets['itemsets'].str.len()
frequent_itemsets.sort_values('support', ascending = False)
```

[6]:
```
     support                            itemsets  length
34  0.238368                     (mineral water)       1
13  0.179709                              (eggs)       1
44  0.174110                         (spaghetti)       1
17  0.170911                       (french fries)       1
9   0.163845                          (chocolate)       1
..       …                                  …     …
0   0.020397                            (almonds)       1
80  0.020264   (frozen smoothie, mineral water)       2
98  0.020131  (whole wheat rice, mineral water)       2
```

```
78  0.020131           (french fries, pancakes)       2
67  0.020131       (cooking oil, mineral water)       2

[103 rows x 3 columns]
```

[7]:
```
frequent_itemsets.groupby('length')['support'].describe()
```

[7]:
```
        count      mean       std       min       25%       50%       75%  \
length
1        53.0  0.062867  0.049074  0.020397  0.027196  0.047460  0.079323
2        50.0  0.029249  0.009154  0.020131  0.022930  0.025997  0.033729

             max
length
1       0.238368
2       0.059725
```

- Use the frequent itemsets to create association rules (take min_threshold as 15%) and evaluate the rules for the following metrics
- – Find top 5 antecedent -> consequent rules based on 'conviction', 'leverage', 'lift'. Explain your findings.
- – Find top 2 & bottom 2 antecedent -> consequent rules based on 'zhang's metric'. Explain your findings.

[8]:
```
rules = association_rules(frequent_itemsets, min_threshold=0.15,␣
↪metric="confidence")
rules
```

[8]:
```
            antecedents      consequents  antecedent support  \
0                (eggs)        (burgers)            0.179709
1            (burgers)           (eggs)            0.087188
2            (burgers)   (french fries)            0.087188
3            (burgers)  (mineral water)            0.087188
4            (burgers)      (spaghetti)            0.087188
..                 …              …                   …
66  (whole wheat rice)  (mineral water)            0.058526
67         (olive oil)      (spaghetti)            0.065858
68          (pancakes)      (spaghetti)            0.095054
69            (shrimp)      (spaghetti)            0.071457
70          (tomatoes)      (spaghetti)            0.068391

    consequent support   support  confidence      lift  leverage  conviction  \
0             0.087188  0.028796    0.160237  1.837830  0.013128    1.086988
1             0.179709  0.028796    0.330275  1.837830  0.013128    1.224818
2             0.170911  0.021997    0.252294  1.476173  0.007096    1.108844
3             0.238368  0.024397    0.279817  1.173883  0.003614    1.057552
```

```
4             0.174110  0.021464    0.246177  1.413918  0.006283    1.095602
..                 …       …           …         …         …           …
66            0.238368  0.020131    0.343964  1.442993  0.006180    1.160960
67            0.174110  0.022930    0.348178  1.999758  0.011464    1.267048
68            0.174110  0.025197    0.265077  1.522468  0.008647    1.123778
69            0.174110  0.021197    0.296642  1.703760  0.008756    1.174209
70            0.174110  0.020931    0.306043  1.757755  0.009023    1.190117

    zhangs_metric
0        0.555754
1        0.499424
2        0.353384
3        0.162275
4        0.320707
..            …
66       0.326080
67       0.535186
68       0.379218
69       0.444850
70       0.462740

[71 rows x 10 columns]
```

[9]:
```
rules.sort_values('conviction',ascending=False).head()
```

[9]:
```
      antecedents      consequents  antecedent support  consequent support  \
62         (soup)  (mineral water)            0.050527            0.238368
53  (ground meat)      (spaghetti)            0.098254            0.174110
59    (olive oil)  (mineral water)            0.065858            0.238368
50  (ground meat)  (mineral water)            0.098254            0.238368
67    (olive oil)      (spaghetti)            0.065858            0.174110

     support  confidence      lift  leverage  conviction  zhangs_metric
62  0.023064    0.456464  1.914955  0.011020    1.401255       0.503221
53  0.039195    0.398915  2.291162  0.022088    1.373997       0.624943
59  0.027596    0.419028  1.757904  0.011898    1.310962       0.461536
50  0.040928    0.416554  1.747522  0.017507    1.305401       0.474369
67  0.022930    0.348178  1.999758  0.011464    1.267048       0.535186
```

[10]:
```
rules.sort_values('leverage',ascending=False).head()
```

[10]:
```
        antecedents      consequents  antecedent support  consequent support  \
52      (spaghetti)    (ground meat)            0.174110            0.098254
53    (ground meat)      (spaghetti)            0.098254            0.174110
64  (mineral water)      (spaghetti)            0.238368            0.174110
63      (spaghetti)  (mineral water)            0.174110            0.238368
50    (ground meat)  (mineral water)            0.098254            0.238368
```

```
     support  confidence      lift  leverage  conviction  zhangs_metric
52  0.039195    0.225115  2.291162  0.022088    1.163716       0.682343
53  0.039195    0.398915  2.291162  0.022088    1.373997       0.624943
64  0.059725    0.250559  1.439085  0.018223    1.102008       0.400606
63  0.059725    0.343032  1.439085  0.018223    1.159314       0.369437
50  0.040928    0.416554  1.747522  0.017507    1.305401       0.474369
```

[11]:
```
rules.sort_values('lift',ascending=False).head()
```

[11]:
```
       antecedents          consequents  antecedent support  \
52    (spaghetti)        (ground meat)            0.174110
53  (ground meat)          (spaghetti)            0.098254
67    (olive oil)          (spaghetti)            0.065858
62         (soup)      (mineral water)            0.050527
41         (milk)  (frozen vegetables)            0.129583

    consequent support   support  confidence      lift  leverage  conviction  \
52            0.098254  0.039195    0.225115  2.291162  0.022088    1.163716
53            0.174110  0.039195    0.398915  2.291162  0.022088    1.373997
67            0.174110  0.022930    0.348178  1.999758  0.011464    1.267048
62            0.238368  0.023064    0.456464  1.914955  0.011020    1.401255
41            0.095321  0.023597    0.182099  1.910382  0.011245    1.106099

    zhangs_metric
52       0.682343
53       0.624943
67       0.535186
62       0.503221
41       0.547490
```

[14]:
```
rules.sort_values('zhangs_metric',ascending=False).head(3)
```

[14]:
```
        antecedents    consequents  antecedent support  consequent support  \
52      (spaghetti)  (ground meat)            0.174110            0.098254
53    (ground meat)    (spaghetti)            0.098254            0.174110
51  (mineral water)  (ground meat)            0.238368            0.098254

     support  confidence      lift  leverage  conviction  zhangs_metric
52  0.039195    0.225115  2.291162  0.022088    1.163716       0.682343
53  0.039195    0.398915  2.291162  0.022088    1.373997       0.624943
51  0.040928    0.171700  1.747522  0.017507    1.088672       0.561638
```

[13]:
```
rules.sort_values('zhangs_metric').head(2)
```

[13]:
```
       antecedents      consequents  antecedent support  consequent support  \
35  (french fries)  (mineral water)            0.170911            0.238368
```

```
38     (spaghetti)   (french fries)            0.174110            0.170911

     support  confidence      lift  leverage  conviction  zhangs_metric
35  0.033729    0.197348  0.827912 -0.007011    0.948894      -0.200452
38  0.027596    0.158499  0.927381 -0.002161    0.985251      -0.086602
```